

A Spotify-Inspired Music Application

JAVASCRIPT WEB DEVELOPMENT ASSIGNMENT

Field	Details
Subject	Web Technology / JavaScript Programming
Assignment Type	Practical Application Development
Technology Stack	HTML5, CSS3, Bootstrap 5, Vanilla JavaScript
Submitted By	
Roll No.	
Class / Division	
Date	
Instructor	Saiprasad Bhise

1. Aim

To design and develop a fully functional, single-page Music Application named "Rhythmix" using HTML5, CSS3, Bootstrap 5, and Vanilla JavaScript — inspired by the Spotify user interface — demonstrating the following core JavaScript concepts:

- UI Design using HTML and CSS with Bootstrap 5 components
- JavaScript-based form validation (Login & Signup) with real-time feedback
- Use of built-in JavaScript dialog functions: alert(), prompt(), and confirm()
- DOM manipulation, event handling, and dynamic content rendering
- Simulated audio playback with progress tracking using setInterval
- Data persistence using localStorage

2. Requirements

2.1 Software Requirements

Component	Technology / Version	Purpose
Browser	Chrome 120+ / Firefox 118+	Runtime environment for the web app
Text Editor	VS Code / Sublime / Notepad++	Writing and editing source code
Bootstrap	Version 5.3.0 (CDN)	Responsive grid, modals, form controls
Bootstrap Icons	Version 1.11.3 (CDN)	Icon set for player and UI controls
Google Fonts	Syne + Space Mono	Custom typography for branding
localStorage API	Built-in Browser API	Persist user data between sessions
JavaScript	Vanilla ES6+ (no frameworks)	Core application logic

2.2 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 or better
RAM	4 GB	8 GB
Storage	100 MB free	500 MB free
Display	1024 × 768	1920 × 1080 (Full HD)

Internet	Required (CDN assets)	Broadband
----------	-----------------------	-----------

2.3 Functional Requirements

- User Registration with full form validation
- User Login with credential verification against localStorage
- Music player with play, pause, skip, shuffle, repeat controls
- Progress bar with seek functionality
- Volume control and mute toggle
- Like/Unlike songs with persistence
- Playlist creation using prompt() dialog
- Add songs to playlists
- Live search across songs, artists, albums
- Genre-based browsing
- Sidebar navigation: Home, Search, Library, Liked Songs

3. Inference / Theory

3.1 JavaScript — Overview

JavaScript (JS) is a lightweight, interpreted, event-driven programming language with first-class functions. Alongside HTML and CSS, it is one of the three core technologies of the World Wide Web. JavaScript enables interactive and dynamic web pages through client-side scripting, requiring no server communication for most user interactions.

Key characteristics of JavaScript include:

- Dynamically typed — variable types are resolved at runtime
- Prototype-based object orientation
- Single-threaded with an event loop (non-blocking via callbacks)
- Access to Web APIs: DOM, localStorage, Fetch, Audio, Canvas, etc.

3.2 HTML5 & Semantic Structure

HTML5 introduces semantic elements (<header>, <main>, <footer>, <aside>, <section>, <nav>) that make web pages accessible and SEO-friendly. Forms in HTML5 support native validation attributes (required, type="email", minlength) which serve as a first line of defence before JavaScript validation.

3.3 Bootstrap 5

Bootstrap 5 is the world's most popular CSS framework. It provides:

- Responsive 12-column grid system based on CSS Flexbox
- Pre-built UI components: Modal, Navbar, Card, Badge, Toast
- Utility-first classes for spacing, colours, sizing, display

- JavaScript plugins: Modal, Dropdown, Tooltip (bundled via bootstrap.bundle.min.js)

3.4 JavaScript Dialog Functions

JavaScript provides three built-in blocking dialog functions that halt script execution until the user responds:

Function	Syntax	Return Value	Use in Rhythmix
alert()	alert(message)	undefined	Welcome message on app load; invalid login warning
prompt()	prompt(message, default)	String (or null if cancelled)	Playlist name input; playlist selection for add-song
confirm()	confirm(message)	Boolean (true/false)	Logout confirmation dialog

3.5 Form Validation in JavaScript

Client-side form validation improves user experience by catching errors before a network request. Two complementary approaches are used in Rhythmix:

HTML5 Native Validation — attributes like required, type="email", minlength on inputs. The form uses novalidate to suppress browser-default bubbles so custom messages can be shown.

JavaScript Custom Validation — each field is validated programmatically using:

- Regular expressions (/^[^\s@]+@[^\s@]+\.[^\s@]+\$/ for email format)
- String.length checks for password strength
- Cross-field comparison (password === confirmPassword)
- Array.some() to check for duplicate email registrations
- Checkbox state (element.checked) for terms agreement

Visual feedback is applied via CSS classes: is-invalid (red border) and is-valid (green border), with custom error messages injected into adjacent <div> elements.

3.6 DOM Manipulation

The Document Object Model (DOM) is a tree-based representation of an HTML document. JavaScript accesses and modifies the DOM to create dynamic interfaces:

- document.getElementById(id) — find single element by ID
- document.querySelector(sel) — CSS selector-based search
- element.innerHTML — set/get HTML content
- element.classList.add/remove/toggle — class manipulation
- element.style.property — inline style changes
- document.createElement() — create new elements programmatically
- parent.appendChild(child) — insert into DOM

3.7 Event-Driven Programming

JavaScript uses an event-driven model where code runs in response to user actions or system events. `addEventListener(type, handler)` is the preferred method as it allows multiple handlers per event and proper cleanup:

- `click` — button presses, card interactions
- `input` — real-time search filtering, password strength checks
- `submit` — form submission interception
- `keydown` — keyboard shortcut (`Space` = play/pause)
- `change` — volume slider
- `blur` — field-level validation on focus loss

3.8 setInterval & Timer-Based Progress

Since the app simulates audio playback without actual audio files, a `setInterval()` timer fires every 500ms to increment the progress bar, update the time display, and detect track completion. `clearInterval()` is called when the user pauses, skips, or the track ends, preventing memory leaks.

3.9 localStorage

`localStorage` is a browser API that allows web apps to store key-value pairs persistently across sessions. It stores only strings, so complex objects are serialized with `JSON.stringify()` and deserialized with `JSON.parse()`. Rhythmix uses it to persist: user accounts, the current logged-in user, liked song IDs, and user-created playlists.

4. Important JavaScript Functions

The following table identifies and explains the key JavaScript functions used in Rhythmix, their purpose, and the concept they demonstrate:

Function Name	Category	Description & JS Concept
<code>init()</code>	Lifecycle	Entry point. Calls <code>loadFromStorage()</code> , all render functions, and sets up event listeners. Demonstrates initialization pattern.
<code>loadFromStorage()</code>	Data Persistence	Reads saved state from <code>localStorage</code> using <code>JSON.parse()</code> . Demonstrates <code>localStorage</code> API and try-catch error handling.
<code>saveToStorage()</code>	Data Persistence	Writes state to <code>localStorage</code> using <code>JSON.stringify()</code> . Called after every state-changing operation.
<code>validateLoginForm()</code>	Form Validation	Validates email format with <code>RegExp</code> and checks password is non-empty. Returns <code>Boolean</code> . Demonstrates custom JS validation.
<code>validateSignupForm()</code>	Form Validation	Validates name length, email uniqueness (<code>Array.some()</code>), password strength, confirm match, and checkbox. Demonstrates multi-field validation.

handleLogin(e)	Event Handler	Prevents default submit, calls validator, authenticates against stored users with <code>Array.find()</code> , and updates UI. Uses <code>alert()</code> for invalid credentials.
handleSignup(e)	Event Handler	Prevents default submit, runs validation, creates new user object, and pushes to users array.
handleLogout()	Event Handler	Uses <code>confirm()</code> dialog for confirmation before clearing <code>currentUser</code> from state and <code>localStorage</code> .
navigateTo(section)	Navigation	Switches visible section by toggling CSS class "active". Updates nav link styles using <code>classList</code> .
playTrack(index)	Player	Loads track metadata into player UI and starts the progress simulation via <code>startProgressSimulation()</code> .
togglePlayPause()	Player	Toggles <code>isPlaying</code> state and starts/stops timer. Demonstrates state management pattern.
nextTrack() / prevTrack()	Player	Advances/retreats index in queue. Handles wrap-around with modulo operator. Supports shuffle via <code>Math.random()</code> .
startProgressSimulation()	Timer	Uses <code>setInterval()</code> to update progress state every 500ms, calculates elapsed time, and triggers auto-next at 100%.
formatTime(seconds)	Utility	Converts seconds integer to MM:SS string using <code>Math.floor</code> and <code>String.padStart</code> . Pure function.
handleSeek(e)	Player	Calculates click position ratio on progress bar to set playback position. Uses <code>getBoundingClientRect()</code> .
toggleLike(songId)	Feature	Adds/removes song ID from <code>likedSongs</code> array using <code>Array.includes</code> , <code>Array.push</code> , <code>Array.filter</code> . Saves to <code>localStorage</code> .
createPlaylist()	Feature	Uses <code>prompt()</code> to get user input for playlist name. Validates input, creates object with <code>Date.now()</code> as unique ID.
addToPlaylist(songId)	Feature	Uses <code>prompt()</code> with numbered list of playlists, validates selection, and pushes <code>songId</code> . Uses <code>confirm()</code> if no playlists exist.
handleSearch(e)	Search	Filters <code>SONGS</code> array using <code>Array.filter</code> and <code>String.includes</code> on multiple fields. Updates queue and renders results.
renderTrackList(songs, id)	Render	Builds HTML string using <code>Array.map</code> and template literals, then injects via <code>innerHTML</code> . Key DOM generation pattern.
filterByGenre(genre)	Feature	Filters songs by genre field, updates queue, navigates to search section, and renders results.
showToast(msg, type)	UI	Dynamically creates and appends a toast div to DOM. Uses <code>setTimeout</code> to auto-remove after 2.8s.
setupPasswordToggle()	UX	Toggles input type between "password" and "text" on button click. Demonstrates input type manipulation.

isValidEmail(email)	Utility	Pure function using RegEx test() to validate email format. Returns Boolean.
setFieldError() / clearFieldError()	Validation	Adds/removes Bootstrap is-invalid/is-valid classes and shows/hides error text. DOM class manipulation.

5. File Structure

The project consists of exactly three files, all in the same directory:

File	Size (approx.)	Role
index.html	~6 KB	HTML skeleton: layout structure, Bootstrap CDN links, Modal markup, script tag
style.css	~8 KB	All custom CSS: CSS variables, sidebar, player bar, cards, animations, responsive
main.js	~15 KB	Complete application logic: data, state, validation, player engine, render functions

6. Program / Implementation

6.1 index.html — Key Sections

The HTML file is structured around a two-panel layout (CSS flexbox): a fixed-width sidebar and a flexible main content area. The main area further splits into a fixed topbar, a scrollable content region, and a fixed player bar at the bottom.

Key HTML elements and Bootstrap classes used:

- d-flex, flex-column, flex-grow-1 — Bootstrap Flexbox utilities for layout
- data-section attribute — custom data attribute for JavaScript navigation
- modal fade, modal-dialog-centered — Bootstrap modal classes
- form-control, is-invalid, invalid-feedback — Bootstrap form validation classes
- novalidate on forms — disables browser validation; JS handles it entirely
- input[type="range"] — HTML5 range input for volume slider

6.2 style.css — Architecture

The CSS is organized into logical sections with comments. CSS custom properties (variables) defined on :root provide a central design token system — changing --accent updates every accent-colored element instantly.

Notable CSS features:

- CSS Grid for quick-picks and genre grids (auto-fill minmax responsive columns)
- CSS Flexbox for player bar, sidebar, track rows
- `::-webkit-scrollbar` for custom scrollbar styling
- `@keyframes slideIn` for toast notification entrance animation
- `backdrop-filter: blur()` on topbar for glassmorphism effect
- `accent-color: var(--accent)` for native range input theming
- `@media (max-width: 768px)` for mobile responsive sidebar collapse

6.3 main.js — Architecture

The JavaScript is written in a structured procedural style using ES6+ syntax. A single state object holds all application data, making it easy to reason about the app. All functions are pure (read/write state only) or clearly labeled as side-effect handlers.

Code is organized into 16 numbered sections:

1. DATA — Mock 20-song library and genre metadata
2. APP STATE — Central mutable state object
3. localStorage HELPERS — load/save functions
4. DOM HELPERS — shorthand \$ and qs functions
5. TOAST — Dynamic notification system
6. FORM VALIDATION — Utility functions and form validators
7. AUTH — Login, Signup, Logout handlers
8. NAVIGATION — Section switching
9. PLAYER ENGINE — Track loading, play/pause, progress
10. TIMER — setInterval-based progress simulation
11. LIKE / UNLIKE — Favourites management
12. PLAYLISTS — prompt()-based creation and management
13. RENDER FUNCTIONS — Dynamic HTML generation
14. SEARCH — Real-time filtering
15. PASSWORD TOGGLE — Input type manipulation
16. EVENT LISTENERS — Central wiring of all UI events

7. Expected Output

Screen / Feature	Expected Behaviour
App Load	alert() welcome message. Home section shows Quick Picks grid, Featured Albums, Trending rows.
Sign Up (valid)	User registered in localStorage. Modal closes. Sidebar shows username. Auth buttons hidden.
Sign Up (invalid)	Red borders on failed fields with specific error messages below each input.
Login (wrong password)	alert() dialog: "Invalid email or password. Please try again."
Playlist Creation	prompt() dialog asks for name. Playlist appears in sidebar and Library

	section.
Logout	confirm() dialog asks for confirmation. On OK, user removed, auth buttons reappear.
Play Track	Player bar updates with song name, artist, emoji. Progress bar animates. Space bar toggles play/pause.
Like Song	Heart icon fills. Toast "Added to Liked Songs". Liked Songs section updates count.
Search	Typing in search box filters 20 songs in real-time, grouped with result count.
Genre Click	Navigates to Search section. Shows all songs of that genre in a track list.
Shuffle ON	Next track picked randomly. Shuffle button glows accent colour.
Repeat ON	Track restarts on completion. Repeat button glows accent colour.
Seek	Click anywhere on progress bar jumps playback to that position.

8. Conclusion

The Rhythmix Music Application successfully demonstrates the practical implementation of core JavaScript and web development concepts in a real-world, production-quality context. The project covers HTML5 semantic structure, Bootstrap 5 responsive design, custom CSS with modern features, and an extensive range of JavaScript programming techniques including:

- DOM manipulation for dynamic content rendering without a framework
- Event-driven programming with `addEventListener` across 20+ event bindings
- Form validation with custom error messages and real-time feedback
- Built-in dialog functions: `alert()`, `prompt()`, and `confirm()` for user interaction
- Timer-based UI updates using `setInterval` and `clearInterval`
- `localStorage` API for client-side data persistence across browser sessions
- Array methods (`filter`, `map`, `find`, `some`, `push`) for data operations
- Object-based state management and modular function design

The project demonstrates that a complex, feature-rich application — competitive in design quality with industry products like Spotify — can be built entirely with vanilla JavaScript and no external libraries, reinforcing a strong foundation in core web technologies before moving to frameworks.

— End of Assignment Document —